

ATP-based Cross-Verification of Mizar Proofs: Method, Systems, and First Experiments

Josef Urban and Geoff Sutcliffe

Abstract. Mizar is a proof assistant used for formalization and mechanical verification of mathematics. The main use of Mizar is in the development of the Mizar Mathematical Library (MML), in which proofs are verified by the Mizar proof checker. The Mizar proof checker has a quite complex implementation, and also lacks the ability to print out detailed atomic proof steps in a format that is easy to verify with an independent proof-checking tool. This can raise concerns about the correctness of the MML. This paper describes a translation of Mizar natural-deduction proofs to the TPTP format used for recording derivations from first-order automated theorem proving systems, and verification of the resulting TPTP format derivations. The system was tested on two nontrivial sets of Mizar problems: the 252 “MPTP Challenge” problems, and 245 Mizar root theorems. The results of these tests are encouraging, and indicate that cross-verification of the whole MML is feasible.

Mathematics Subject Classification (2000). Primary 03E99; Secondary 68T15.

Keywords. Formalized mathematics, automated reasoning, verification.

1. Introduction, motivation, and related work

Mizar [17, 19] is a proof checker based on first-order classical logic and set theory. The main use of Mizar is in the development of the Mizar Mathematical Library (MML)¹, a large library of formally verified mathematics, allowing the formalization of more and more advanced mathematical theories.

1.1. Motivation: Verification of Mizar proofs

Starting in the 1990s with John Harrison’s Mizar Mode for HOL [6], the ideological gap between Mizar and other proof assistants has begun to diminish, and various

Supported by a Marie Curie International Fellowship within the 6th European Community Framework Programme.

¹<http://mizar.org/library/>

versions of the Mizar declarative proof formalism have been developed [26, 37, 39]. However, so far, there has been no independent verification of MML proofs. While Mizar’s reasoning formalism (often said to be derived from Jaskowski-style natural deduction [7, 14]) is quite simple and intuitive, it is Mizar’s checking of “atomic inference steps” (denoted in Mizar proofs with the keyword **by**) that makes Mizar emulation complicated. Although the basic idea motivated by the notion of obviousness [3, 18] is relatively easy to explain [38], and has even been re-implemented in Harrison’s Mizar Mode for HOL, there have been a number of additions to Mizar, e.g., interaction with Mizar’s type system, built-in computer algebra [10, 11], etc., that make it quite expensive to produce detailed documentation of Mizar’s proof checking.² On the other hand, Mizar’s classical first-order foundations are very close (probably closer than any other well-known proof assistant) to the formalism used in today’s efficient Automated Theorem Proving (ATP) systems. Although the Tarski–Grothendieck set-theoretical foundations of Mizar [28, 29]) uses an infinite axiomatization – the Fraenkel replacement scheme, the number of first-order instances of such schemes in any particular Mizar proof is finite.³

Translation of the MML to ATP formats, in particular the TPTP format [23], is the goal of the Mizar Problems for Theorem Proving (MPTP) project. The MPTP system has already been used for testing ATP systems’ abilities to reprove long MML proofs (sometimes containing hundreds of atomic inference steps) from scratch, i.e., without any knowledge of the MML proof [33]. This has been done mainly to measure the potential of ATP systems as proof assistance tools for Mizar authors, and the results are quite acceptable – about 40% of Mizar theorems not involving arithmetic have been reported as reprovable by ATP systems in this high-level way [33]. (This number has grown quite significantly recently: as of December 2007 it is about 60%, thanks to the use of stronger versions and combinations of ATP systems, and better preparation of ATP problems by the MPTP system.) However, there is no hope for using this relatively crude method for verification of the whole MML. Other MPTP-based experiments [31, 33] have shown that current ATP systems are able to verify an overwhelming majority of atomic inference steps in the MML, without any low-level guidance – more than 99% of 18,429 atomic inference steps in 48 initial Mizar articles, 92% within a 4s time limit (on 3GHz Xeons) [33]. In [30], such proofs of atomic inference steps are used to explain the steps, complementing the explicit higher-level structure of Mizar articles. Given this capability for the verification of atomic inference steps, the next logical step (described in this paper) is to develop techniques extending this to ATP-based

²The first author’s unfinished efforts in this direction almost doubled the size of the Mizar proof checker code. This difficulty with documenting proofs is not unique: one reason Bill McCune gave (in a personal discussion) for writing the Prover9 system was the necessity of redesigning Otter after the addition of code to produce detailed proofs.

³It would probably be possible and quite easy to replace the infinite ZF-like Mizar axiomatics with finite NBG-like axiomatics (as often used with ATP systems, e.g., in [16]). However, trading several targeted first-order instances of replacement for the general purpose NBG “set formation rules” is likely to degrade ATP system performance in verification and reproving tasks.

verification of whole (sometimes quite long) Mizar proofs, with the potential target of ATP-based verification of the whole MML.

This paper describes how the MPTP translation has been extended (Section 2.2) so that both the atomic inferences and the higher-level natural deduction reasoning structure of MML proofs can be verified, using the GDV verification tool [25] (Section 3) for verification of ATP systems' derivations. The higher-level structure (Section 2.2) is used to effectively “glue together” verified atomic inference steps, in a way quite similar to their “gluing together” in MML proofs. To achieve this, the TPTP format has been extended for encoding derivations containing *assumptions* (Section 2.1), correctness conditions for such derivations have been designed and implemented in the GDV tool, and an explanation of the Mizar natural deduction constructs has been provided in terms of this simpler formalism (Section 2.2). The result is a system for the independent verification of MML proofs.

1.2. Related verification work

A list of previous mutual proof assistant verifications would probably be quite long (the translations between HOL Light and Isabelle are mentioned below). However, they are usually done for systems with low-level “proof object” capabilities, or systems in which the addition of such capability is relatively easy, thanks to their “minimal logical kernel” design. The Otter and Prover9 ATP systems can produce detailed proof objects, and the proof objects can be verified by the IVY system [9]. A recent notable use of ATP systems for certification of safety properties of NASA software is described in [4].

The rigorous translation of MML constructs into classical first-order constructs, as has been done in this work, provides a basis for trusted semantic communication between the MML and other systems, e.g., proof assistants such as HOL and Isabelle. Such inter-system communication is useful for very large formalization efforts, e.g., the verification of the proof of the Kepler Conjecture [5]. In this context, communication between LCF-based systems has recently become popular [8, 13].

The extension of the TPTP format for encoding derivations containing assumptions now allows for more natural verification of derivations from refutation-based ATP systems. It is planned to convert the checking of the negated conjectures in refutations to a checking-of-assumptions form.

1.3. Targets of the verification

1.3.1. The MPTP challenges. This is a set of 252 related mathematical problems translated from the Mizar library by the MPTP system, to create a benchmark for measuring the capability of ATP systems to work in large theories.⁴ The conjectures of the problems are the MML theorems that were recursively used for

⁴<http://www.cs.miami.edu/~tptp/MPTPChallenge/>

the proof of one half (one of two implications) of the general topological Bolzano-Weierstrass theorem. The whole problem set contains 1234 formulas (the 252 conjectures plus supporting definitions, etc.) and 418 symbols. In the *bushy* division of the challenge, the goal is to reprove the Mizar theorems from their explicit Mizar references (and some background formulas used implicitly by Mizar, added by the MPTP system), while the problems in the *chainy* division contain all the “previous knowledge” as lemmas. This results in an average chainy problem size of ca. 400 formulas, and the problems are quite hard for currently existing ATP systems. As the chainy versions of the problems are created by addition of redundant lemmas to the bushy versions, it is sufficient to verify only the MML proofs of the less difficult bushy division problems. Still, some of these problems have quite long Mizar proofs, making their direct solution by ATP systems quite difficult. As of May 2007, 199 of the 252 problems (79%) have been solved by any ATP system,⁵ leaving open questions about the provability of the remaining 53 problems. Since errors have been encountered before, in both the Mizar verifier and the MPTP translation, an independent verification of the problems’ MML proofs would be useful. This has been quite successfully carried out, and is discussed in Section 4.1.

1.3.2. MML root theorems. As the verification of the MML proofs of the relatively hard MPTP Challenge problems turned out to be feasible, the next step was to test the verification process on the MML proofs of “very hard” MML theorems. For this purpose 16 root (i.e., unused by others) MML articles were chosen, and from them all of their root theorems. These 245 theorems are therefore at the roots of the DAG formed by all (ca. 40000) MML theorems, giving high chance that all the Mizar mechanisms are used fully for their Mizar proofs. The difficulty of solving these problems directly was confirmed by testing with the strongest system (Fampire⁶) from the bushy division of MPTP Challenge. While Fampire solves 75% of the MPTP Challenge problems in 300s, it solves only 51% of the 245 root problems in 300s. The verification of these problems’ MML proofs is discussed in Section 4.2.

2. Translation of Mizar proofs to TPTP derivations

2.1. Extending the TPTP format for assumptions

Assumptions (*suppositions* in Jaskowski’s [7] original terminology) are unproved propositions introduced (assumed to be true) locally at some part of a proof, coupled with an elimination (discharge) rule for making the results of such subproofs unconditionally valid. A common implicit use of these introduction/discharge rules in refutational ATP systems is the introduction of the negated conjecture, the derivation of a contradiction, and the discharge of the negated conjecture at that point, i.e., the argument that “ $\sim \text{conjecture} \vdash \text{\$false}$ ” is equivalent to “ $\vdash$

⁵See <http://www.cs.miami.edu/~tptp/MPTPChallenge/Results/SVGResults.html>

⁶Vampire [20] using FLOTTER [12] as a clausifier

$\sim \text{conjecture} \Rightarrow \false ”, and hence to “ $\neg \text{conjecture}$ ”. Other examples from the ATP world include explicit (SPASS-like) splitting [36], and the formalisms used in tableaux systems.

In order to record information about the use of assumptions in derivations, a new formulae role **assumption** was added to the TPTP language, and a convention was established for recording a list of assumptions in the source information of an inferred formula, to keep track of the assumptions upon which the formula depends. A convention was also established for recording the discharge of assumptions, making it possible to correctly argue that a derivation is independent of the assumptions introduced in it. An example of an assumption introduced into a derivation, an inferred formula that depends on recorded assumptions, and an inference involving the discharge of assumptions (modified from an MPTP Challenge problem), are:

```
fof(e1, assumption, subset(c1, c2), introduced(assumption)).

fof(i3, plain, c2 = set_union2(c1, set_diff2(c2, c1)),
    inference(conclusion, [status(thm), assumptions([dt_c1, dt_c2, e1])],
        [e3, i4])).

fof(i2, plain, (subset(c1, c2) => c2 = set_union2(c1, set_diff2(c2, c1))),
    inference(discharge_asm, [status(thm), assumptions([dt_c1, dt_c2]),
        discharge_asm(discharge, [e1])], [e1, i3])),
```

This “recording” approach is more effective than adding assumptions as antecedents of implications to their descendants. Adding assumptions as antecedents would significantly increase the difficulty of the ATP problems built from the Mizar atomic inference steps, possibly preventing completion of the cross-verification. The recording approach is sufficient for translation of Mizar proofs into efficiently verifiable TPTP derivations (see Section 2.2). The consequence of this decision is a need to ensure that recorded assumptions are correctly propagated and discharged. This is done by GDV, as described in Section 3. These extensions to the TPTP language have also been adopted in other TPTP tools, e.g., IDV [35] and AGInTRater [15].

2.2. Mizar natural deduction

Quite often it is claimed that Mizar uses Jaskowski-style natural deduction. Even though Jaskowski’s formalism is certainly related to the Mizar formalism, there are too many differences to take Jaskowski’s original paper [7] (or its more accessible version [14]) as an accurate account of the Mizar formalism that could be used for explaining its translation to classical first-order logic form. Quite a precise overview of the Mizar formalism is given in [6], for the purpose of its implementation in HOL. In what follows the use of Gentzen-like introduction/elimination terminology, and comparison with various Gentzen-style calculi, is avoided. This is mainly to keep the exposition simple, and to avoid possible confusion caused by different flavors and semantics of those calculi used in different systems (see [14] for a detailed overview of these issues).

The Mizar parser translates Mizar articles into a semantic XML format⁷ [32] that is used internally by the Mizar proof checker, and also by various external tools such as the MPTP translator. The XML format is specified in the RELAX NG schema language. The XML format specification is always up-to-date, because it is generated in a semi-automated way from the Mizar source code, and because XML validation can be used to check the XML representation of articles. For these reasons, the XML format specification has been used in this work as an up-to-date definition of the Mizar formalism. The part of the XML format specification for the Mizar natural deduction proofs is (simplified) as follows:

```
## Theorem as a proposition with justification.
JustifiedTheorem = element JustifiedTheorem {Proposition, Justification}
Justification = ( Inference | Proof | SkippedProof )
## By and From encode the atomic inference steps done by Mizar.
Inference = ( By | From | element ErrorInf { empty } )
## By encodes one simple justification from zero or more references.
By = element By { Ref* }
## From encodes a simple justification involving Mizar schemes
From = element From { Ref* }
## Proofs (of BlockThesis) encode ND reasonings consisting of many steps.
Proof = element Proof {BlockThesis, Reasoning }
## Reasoning is a series of skeleton and auxiliary items (steps)
## optionally finished by reasoning per cases.
Reasoning = ( ( SkeletonItem | AuxiliaryItem )*, PerCasesReasoning? )
## Skeleton items are the steps that change the current thesis, the
## new Thesis is printed explicitly after them.
SkeletonItem = ((Let|Conclusion|Assume|Given|Take|TakeAsVar), Thesis)
## Auxiliary items are items which do not change thesis.
AuxiliaryItem=(JustifiedProposition|Consider|Set|Reconsider|DefFunc|DefPred)
```

The translation of MML proofs in this format, to the TPTP format, depends on the type of justification/inference used in each MML step. Generally the translation can add parent formulas that encode “background” information used implicitly by the Mizar proof checker, e.g., the Mizar type hierarchy.

The **By** and **From** justifications denote atomic inference steps that are simple enough for the Mizar proof checker. There is no substructure to atomic inference steps, and they are translated directly as first-order inferences, as illustrated here (the Mizar inferences on the left translate into the TPTP format derivation on the right).

```
A1: p;                fof(a1,plain,p).
A2: p implies q;      fof(a2,plain,p => q).
A3: q by A1, A2;      fof(a3,plain,q,
                    inference(mizar_by,[status(thm)], [a1,a2])).
```

A full **Proof** is used when the Mizar proof checker is not strong enough to check the step as a **By** or **From** construct. In this situation the goal (thesis in Mizar terminology) either needs to be simplified by various natural deduction steps (**Let**, **Conclusion**, **Assume**, **Given**, **Take**, **TakeAsVar**, **PerCasesReasoning**),

⁷See <ftp://mizar.uwb.edu.pl/pub/version/doc/xml/Mizar.html>

or the proof context needs to be augmented by a new lemma (**JustifiedProposition**), a new constant satisfying some property (**Consider**, **Reconsider**), or a useful macro (**Set**, **DefFunc**, **DefPred**). The following subsections explain these parts of Mizar reasoning in more detail. In all cases Mizar uses a *normalized form* of the **thesis** – see, e.g., [38] for details.

2.2.1. Explanation of the Mizar assumption and conclusion steps.

```

:: Example 1 (Mizar code)
Lemma1: p implies q;           :: this has been proved already
Lemma2: q implies r;           :: this has been proved already
Goal1: p implies q & r
proof                           :: (Proof) thesis (Goal1) is: p implies q & r
  assume E1: p;                 :: (Assumption) thesis (T1) is: q & r
  thus E2: q by Lemma1,E1;       :: (Conclusion) thesis (T2) is: r
  thus E3: r by Lemma2,E2;       :: (Conclusion) thesis (T3) is: verum ($true)
end;

:: Example 2 (Mizar code)
Lemma3: p implies q;           :: this has been proved already
Lemma4: not q;                 :: this has been proved already
Goal2: not p
proof                           :: (Proof) thesis (Goal2) is: not p
  assume E1: p;                 :: (Assumption) thesis (T1) is: contradiction
                                :: ($false)
  E2: q by E1, Lemma3;          :: (JustifiedProposition) thesis unchanged
  thus E3: contradiction        :: (Conclusion) thesis (T2) is: verum ($true)
    by E2, Lemma4;
end;

```

An **Assume** (keyword **assume**) step is used when the thesis can be considered to be an implication. Very much like in the original Jaskowski formalism, it adds the antecedent (**p** in Example 1) of the **thesis** (**p implies q & r**) to the list of valid propositions available for further use in this block, and changes the **thesis** of the block to its consequent (**q & r**). As can be seen from Example 2, the Mizar notion of normalization is quite strong. It identifies the proposition “**not p**” with the proposition “**p implies contradiction**”. In this feature Mizar certainly differs from Jaskowski’s original treatment. The advantage is that it turns a proof by contradiction into just a special case of proof by supposition.

A **Conclusion** (keyword **thus**) step is used to prove (a part of) the current thesis. Mizar checks that the proposition that is proved is either equal to the current thesis (the **E3** case in Example 1), or a conjunct of the current thesis (the **E2** case in Example 1). This way, the thesis is reduced by each **Conclusion** step, and upon encountering the **end** keyword, Mizar checks that the thesis is just **verum** (**\$true**). It should be noted that here again the Mizar normalization is responsible for seemingly unrelated deconstructions of the thesis. For example, if the thesis is “**p iff q**”, a **Conclusion** step proving “**p implies q**” is legal, transforming the thesis to “**q implies p**”.

2.2.2. Translation of the Mizar assumption and conclusion steps.

```

%% Translation of Example 1 into a TPTP derivation
fof(lemma1,plain, p => q).
fof(lemma2,plain, q => r).
fof(e1,assumption,p,introduced(assumption)).
fof(e2,plain,q,
  inference(mizar_by,[status(thm),assumptions([e1]),[lemma1,e1])).
fof(e3,plain,r,
  inference(mizar_by,[status(thm),assumptions([e1]),[lemma2,e2])).
fof(t3,plain,$true,introduced(tautology)).
fof(t2,plain,r,
  inference(conclusion,[status(thm),assumptions([e1]),[t3,e3])).
fof(t1,plain,q & r,
  inference(conclusion,[status(thm),assumptions([e1]),[t2,e2])).
fof(goal1,plain,p => q & r,
  inference(discharge_asm,[status(thm),discharge_asm(discharge,[e1]),
    [e1,t1])).

```

The translation of these Mizar proof steps to TPTP format is quite straightforward. In this TPTP encoding of Example 1, note that the **theses** are collected in a backwards manner and used to justify their **thesis**-predecessors, either by a **conclusion** inference (usually involving some other lemmas), or by a **discharge_asm** inference involving an assumption. This ends at the top-most thesis, i.e., the goal of the proof. The bottom-most **\$true** thesis could be trivially removed, but retaining it makes the translated proof correspond more faithfully to its Mizar counterpart. Note how the assumptions are propagated and finally discharged.

2.2.3. Explanation of the Mizar let, consider, and take steps.

```

:: Example 3 (Mizar code)
Lemma1: for x being set holds p(x);           :: already proved
Lemma2: for x being set holds p(x) => q(x);    :: already proved
Goal1:  for x being set holds q(x)             :: (Proof) thesis (Goal1) is:
proof                                           :: for x being set holds q(x)
  let new_const1 be set;                       :: (Let) thesis (T1) is: q(new_const1)
  E1: p(new_const1) by Lemma1;                 :: (JustifiedProposition) thesis unchanged
  thus E2: q(new_const1) by Lemma2,E1;         :: (Conclusion) thesis (T2)
end;                                           :: is: verum ($true)

:: Example 4 (Mizar code)
Lemma3: ex x being set st p(f(x));             :: already proved
Lemma4: for x being set holds p(x) => q(x);    :: already proved
Goal2:  ex x being set st q(x)                 :: (Proof) thesis (Goal1) is:
proof                                           :: ex x being set st q(x)
  consider new_const2                          :: (Consider) thesis unchanged,
  being set such that E1:                      :: adds new_const2 of type set,
  p(f(new_const2)) by Lemma3;                 :: and p(f(new_const2))
  take f(new_const2);                         :: (Take) thesis (T1) is:
                                              :: q(f(new_const2))
  thus E2: q(f(new_const2))                   :: (Conclusion) thesis (T2) is:
    by E1, Lemma4;                           :: verum ($true)
end;

```


A **Let** (keyword `let`) step is used when the thesis is a universally quantified formula. This step introduces a new constant of the proper type (`new_const1` of type `set` in Example 3), which is available from then on in the proof block. Mizar numbers the new constants serially, thus guaranteeing that they are really new. The thesis is instantiated with this new constant (to `q(new_const1)` in Example 3). This corresponds to the common mathematical expression “let’s have an arbitrary but fixed x ”, and provided that the constant is fresh (which is guaranteed in Mizar by the serial numbering), the original universally quantified thesis follows from the instantiated thesis. (This is the standard theorem about constants in Hilbert’s predicate calculus.)

A **Consider** (keyword `consider`) step has no effect on the proof thesis, although it is in some sense dual to the **Let** construct. Given an existentially quantified statement (like `Lemma3` in Example 4), Mizar creates a fresh constant of the appropriate type (`new_const2` in Example 4) and instantiates the existential statement with it. The new constant and the instance are available for further use in the proof block. Provided that the constant is fresh, this is again just a conservative extension of the original theory.

The **Take** (keyword `take`) step is used to select a suitable term for instantiating an existentially quantified thesis (`f(new_const2)` is taken in Example 4). The type of the term has to correspond to the type of the existential variable. The original thesis then follows from the instantiated one. A minor variant of this syntax is the **TakeAsVar** step (in Example 4 it would be “`take new_const3 = f(new_const2);`”), which additionally creates a fresh constant of the same type as the selected term, adds their equality to the proof context, and instantiates the thesis with this fresh constant.

2.2.4. Translation of the Mizar let, consider, and take steps.

```
% Translation of Example 3 into a TPTP derivation
fof(lemma1,plain, ![X]: (set(X) => p(X))).
fof(lemma2,plain, ![X]: (set(X) => (p(X) => q(X)))).
fof(henkin_ax1, plain, (set(new_const1) => q(new_const1)) =>
    ![X]: (set(X) => q(X)),
    introduced(definition, [new_symbol(new_const1)]))).
fof(type_ass1, assumption, set(new_const1), introduced(assumption)).
fof(e1,plain, p(new_const1),
    inference(mizar_by,[status(thm),assumptions([type_ass1]]),
    [type_ass1,lemma1])).
fof(e2,plain, q(new_const1),
    inference(mizar_by,[status(thm),assumptions([type_ass1]]),
    [type_ass1,e1,lemma2])).
fof(t2,plain, $true,introduced(tautology)).
fof(t1,plain, q(new_const1),
    inference(conclusion,[status(thm),assumptions([type_ass1]]),[t2,e2])).
fof(t1_1,plain, set(new_const1) => q(new_const1),
    inference(discharge_asm, [status(thm),
        discharge_asm(discharge, [type_ass1]]), [type_ass1, t1])).
fof(goal1,plain, ![X]: (set(X) => q(X)),
    inference(let, [status(thm)], [t1_1, henkin_ax1])).
```

```

%% Translation of Example 4 into a TPTP derivation
%% To simplify the explanation, we add the following axiom, known to
%% Mizar implicitly.
fof(f_is_set, axiom, ![X]: set(f(X))).
fof(lemma3,plain, ?[X]: (set(X) & p(f(X)))).
fof(lemma4,plain, ![X]: (set(X) => (p(X) => q(X)))).
fof(henkin_ax2, plain, (?[X]: (set(X) & p(f(X))) =>
    (set(new_const2) & p(f(new_const2)))),
    introduced(definition, [new_symbol(new_const2)])).
fof(e0,plain, ?[X]: (set(X) & p(f(X))),
    inference(mizar_by,[status(thm)],[lemma3])).
fof(e1,plain, set(new_const2) & p(f(new_const2)),
    inference(consider,[status(thm)],[e0, henkin_ax2])).
fof(e2,plain, q(f(new_const2)),
    inference(mizar_by,[status(thm)],[e1, lemma4])).
fof(t2,plain, $true,introduced(tautology)).
fof(t1,plain, q(f(new_const2)),
    inference(conclusion,[status(thm)],[t2, e2])).
fof(goal2,plain, ?[X]: (set(X) & q(X)),
    inference(take, [status(thm)], [t1, f_is_set])).

```

The translation of Example 3 demonstrates the need for dealing with Mizar’s typed language [1], which is done through relativization to the untyped formalism used by ATP systems. The method of translating the **Let** inference step is the introduction of a Henkin (sometimes also called Skolem) axiom (**henkin_ax1** above) partially defining the new constant. These additional axioms are generated by the MPTP translation system from the known (instantiated) thesis (say “**t(constant)**”) as the corresponding implications (“**t(constant) => ![X]:t(X)**”). Axioms of this form (as well as of the dual form used for translation of the **Consider** constructs, i.e., “**?[X]:t(X) => t(constant)**”) extend the existing theory conservatively, provided that the constant is really new. To check that Mizar always generates a fresh constant, the “**introduced(definition, [new_symbol(...)])**” TPTP source is used for these axioms. This tells GDV to carry out this check independently (see the description of leaf checks in 3.2). GDV does not yet implement a structural check verifying that these are really Henkin axioms, i.e., that they have the required form of an instance implying the universal formula (or of the dual existential form). Their correctness is “by construction” in the MPTP translation system, which is quite obvious from the following core of the Prolog code used for this:

```

%% given a Constant, and the Instance, generate a Henkin
%% axiom introducing Const
create_henkin_axiom_let(+Const,+Instance,-Henkin_Ax):-
    apply_const_subst((Const/NewVar), Instance, GenInst),
    Henkin_Ax = (Instance => ( ! [NewVar] : GenInst )).

%% replace atom Const everywhere in Fla with Var, yielding Fla1
apply_const_subst((Const/Var),+Fla,-Fla1):-
... [code skipped] ...

```

Given that the relativization method is used for type translation, **Let** steps additionally have to be complemented by the assumption and corresponding discharge of the new constant's type (see the translation of Example 3).

The translation of a **Consider** construct is very similar to **Let** – again a corresponding Henkin axiom is added by the MPTP translation system, but the constant's type is derived instead of assumed. The translation of **Take** (or **TakeAsVar**) steps is quite straightforward, as they only justify an existential thesis from its instance (which is a standard first-order inference). It has to be proved however that the “taken” term has the proper type, required by the existentially quantified goal (see the translation of Example 4, where the `f_is_set` axiom used implicitly by Mizar had to be added explicitly).

2.2.5. Explanation and translation of the rest of the Mizar reasoning constructs.

The remaining Mizar reasoning constructs not explained so far are: **Given**, **PerCasesReasoning**, **JustifiedProposition**, **Reconsider**, **Set**, **DefFunc**, and **DefPred**. The **Set**, **DefFunc**, and **DefPred** constructs define macros (shorthands for terms and formulas) that are automatically expanded during the translation of formulas, and therefore the macros themselves do not have to be translated.⁸ A **JustifiedProposition** is an arbitrary lemma introduced in the reasoning, and it (similarly to **JustifiedTheorem**) can be justified either by atomic inference steps, or by its own subproof. A **Given** step is a compressed syntax for an **Assume** step followed by a **Consider** step. **PerCasesReasoning** allows several logically complementary alternatives to be explored in Mizar, and it is translated using the TPTP assumption format. **Reconsider** is used to justify that a term has a certain type, and it introduces a new constant (with the new type) as a shortcut for the term. This is translated as a corresponding equality introducing the new constant (its freshness is again checked by GDV), and a (provable) proposition stating the typing of the new constant. The translation of all these constructs does not require any more extensions of the TPTP formalism, or any new kinds of axioms, and given the explanations in the previous sections, their translation becomes quite routine. Interested readers can consult the Mizar TPTP web page⁹ (explained in more detail in [35]), where all (ca. 40000) Mizar theorems are linked to their corresponding TPTP derivations.

3. The GDV derivation verifier

GDV [25] is a tool for verifying derivations in the TPTP format. GDV uses structural and semantic techniques. Structural verification checks that inferences have been done correctly in the context of the derivation, e.g., checking that the derivation is acyclic. Semantic verification checks the required semantic properties of inference steps, e.g., checking that an inferred formula is a logical consequence of its parents. As [25] already provides comprehensive coverage of the design and

⁸Some ATP systems' clausifiers introduce similar shortcuts to improve the CNF.

⁹<http://www.cs.miami.edu/~tptp/MizarTPTP/>

implementation of many aspects of GDV, this section provides an overview of the GDV process, and details of new aspects of GDV that were used in this work.

3.1. Structural verification

The structural verifications done by GDV are, for the most part, quite simple. The most basic checks are made first: checking that all the nodes in the derivation DAG are distinctly named, checking that all inference steps are adequately documented and the named parents exist, checking that the derivation is acyclic, i.e., the derivation is a DAG, and checking that refutations have (only) *false* roots. More complex structural checks are made after that: checking that proofs by contradiction are well formed, checking that split refutations are not mutually dependent, and checking that inference chains with assumptions have been recorded correctly.

Other than the checking of assumptions, all the structural checks listed are described in [25]. The checking of inference chains with assumptions was added to GDV for this work – it was not needed previously because GDV had been used only to check derivations from refutation based ATP systems, which do not use assumptions. The structural checking of inference chains with assumptions has two aspects. First it is necessary to check that assumptions are propagated from parents to inferred formulae, modulo discharged assumptions. This is done by ensuring that the set of assumptions of the parents, i.e., the parents that are assumptions plus the recorded assumptions of the parents, are a subset of the union of the discharged and recorded assumptions of the inferred formula. Second it is necessary to check that all assumptions are discharged. This is done by checking that no root of the DAG has any remaining assumptions. Note that these structural checks on assumptions make no examination of the logical formulae involved, so that they do not ensure that the discharges correctly remove dependence on the assumptions. Semantic verification is used to check that, as is described in Section 3.2.

3.2. Semantic verification

The technique used in semantic verification is to encode the required semantic relationship between each inferred formula and its parent formulae into logical obligations, in the form of ATP problems. The obligations are discharged by having trusted ATP systems solve the ATP problems. The required semantic relationship between an inferred formula and its parent formulae depends on the intent of the inference rule used. Most commonly an inferred formula is intended to be a theorem (logical consequence) of its parent formulae, but in other cases, e.g., skolemization and splitting, the inferred formula has a weaker relationship with its parents. The intent is recorded as an SZS [27] status in the inference record of each inferred formula in a TPTP format derivation, e.g., in the second formulae in Section 2.1 the SZS status is `thm`, which indicates that the inferred formula is required to be a theorem of its parents. GDV's semantic verification has three phases: leaf verification, rule specific verification, and general semantic verification.

There are two types of leaves in a derivation – leaves that come from the problem solved by the derivation, and leaves that were introduced by the ATP system.

Verification of leaves that come from the problem is described fully in [25], based on the idea that axiom leaves should be derivable from problem axioms, and a conjecture leaf be able to derive the problem conjecture. Leaves are introduced into derivations by ATP systems for a variety of (legitimate) reasons. For this work the reasons that occur are the introduction of tautologies, the introduction of definitions, and the introduction of assumptions. Tautologies are verified by proving them. Definitions and assumptions do not produce a verification obligation, but definitions are checked to ensure that the defined symbol does not occur in any other non-introduced leaf.

Rule specific semantic verification deals with explicit splitting, as implemented in the ATP system SPASS [36], pseudo-splitting, as implemented in Vampire [20] and E [21], application of definitions as implemented in E, and discharging assumptions as found in the derivations that are the focus of this work. The verification of splitting is dealt with in [25], and the application of definitions does not arise in this work. To verify that assumptions have been discharged, the assumptions must be added as conditions on the non-assumption parents, forming an implication from the conjunction of the assumptions to the parents. The inferred formula must then be proved (the SZS status must be `thm`) from this implication. In GDV this overall verification obligation is broken down into an equivalent set of small verification obligations, to prove the inferred formula from each of the negated assumptions, and from the non-assumption parents. The benefit of breaking down the verification into the individual components is that a failure to discharge any of the obligations provides precise information regarding the fault in the derivation.

General semantic verification has not been modified from that described in [25]. The most common type of verification obligation that must be discharged is, as explained above, to prove that the inferred formula is a theorem of its parents (SZS status `thm`). While this can always be directly attempted by a trusted ATP system, depending on the nature of the inference step the obligation problem might be quite difficult, and the discharge attempt might fail due to resource limits. In these cases GDV uses model finding ATP systems to try to show that the inferred formula is countersatisfiable wrt its parents (SZS status `csa`), i.e., the obligation cannot be discharged, thus establishing that the inference is really faulty. Two other types of obligations that arise are to prove that the inferred formula is a countertheorem of its parents (SZS status `cth`), e.g., in the case of a negated conjecture, and to prove a satisfiability-bijection between the parents and inferred formula, e.g., in skolemization steps. At this stage GDV does only incomplete verification of satisfiability-bijection inferences. Such steps are therefore avoided in the translation of Mizar proofs, and the additional Henkin axioms are used instead for Mizar steps (`Let`, `Consider`), analogous to ATP skolemization.

The verification provided by GDV is dependent on trust in the underlying ATP systems used by GDV. The simpler the trusted systems, the less likely that there are bugs. At the same time, simpler systems are less powerful, and may not be able to discharge the obligation problems. If the underlying system is not

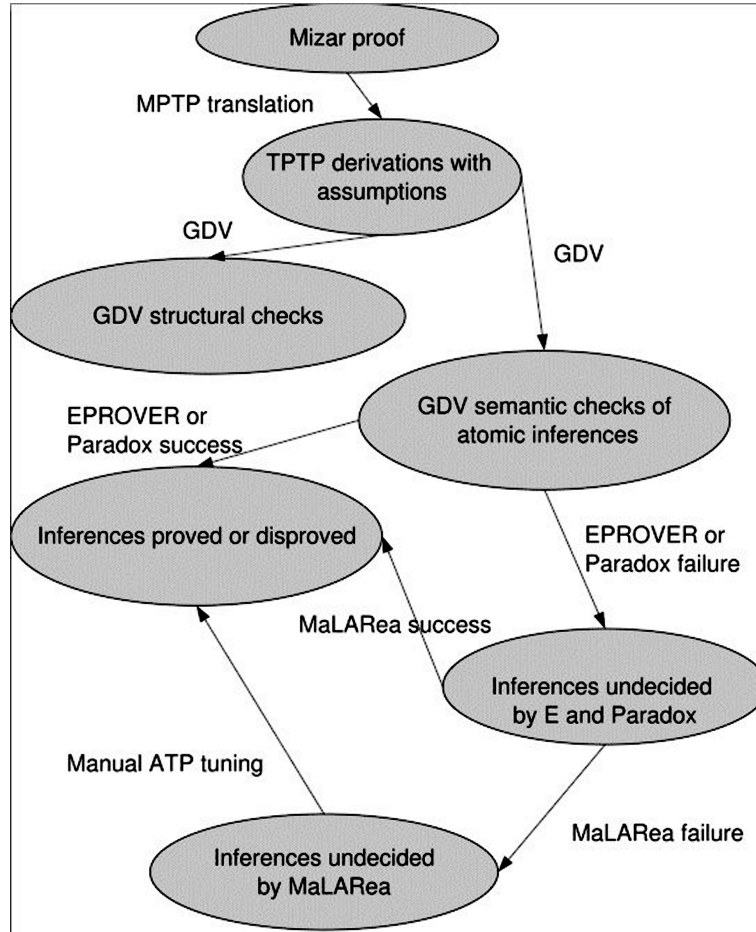


FIGURE 1. Overview of the systems used for verification.

trusted, GDV can be used to verify the underlying system's proofs, using a more trusted ATP system. For example, in Section 4 E 0.99 [21] is used in GDV for the verification of the MML proofs; if E were not trusted, a system such as Otter (for which there is much empirical evidence of trustworthiness) may be used in GDV to verify E's proofs.

4. Verification of the target problem sets

Figure 1 provides an overview of the verification process, and the component software tools used. This configuration was used to verify the MML proofs of the targets described in Section 1.3.

4.1. Verification of the MPTP challenge problems

The MPTP system was used to generate the derivations corresponding to the 252 problems in the MPTP Challenge. The derivations, together with the corresponding MPTP Challenge problems, were given to GDV for verification. GDV was configured to use E 0.99 [21] for finding proofs, and Paradox 2.0b [2] for finding models. A 10s time limit was imposed on each E and Paradox run. All the structural checks (notably the propagation of assumptions) were successful, verification of leaves (checking that they either correspond to the problem, or are introduced definitions, tautologies, or assumptions) was successful, and all the (1303) ATP problems originating from checking the correctness of discharging assumptions were successfully proved.

From the 6765 ATP problems corresponding to the Mizar atomic inference steps, 60 could not be solved in the 10s time limit by E, but none were found to be countersatisfiable by Paradox. The 60 unsolved ATP problems came from 27 derivations, i.e., at this point $252 - 27 = 225$ of the problems' MML proofs were verified. Such timeouts can often be resolved by heuristically pruning the (potentially superfluous) background formulas added as axioms by the MPTP translation. Therefore the MaLARea system, developed for doing such heuristic pruning along with time limit manipulations in a systematic and automated way [34], was used. To create a wide training set for MaLARea's machine learning based heuristic pruning of axioms, all 759 of the ATP problems generated by GDV during verification of the 27 derivations were put into MaLARea's initial set of problems. MaLARea was then run with its maximal time limit set to 64 seconds, the maximal axiom limit set to 64 axioms, using the E and SPASS provers (these values were estimated empirically). The result of the MaLARea run was the solution of 742 of the 759 problems (this took about 4 hours), solving 43 of the 60 previously unsolved ATP problems, and leaving 17 ATP problems unsolved. This was a sufficiently low number to inspect them manually.

One ATP problem contained quite large formulas, and could be solved easily by running E with more aggressive introduction of definitions during clausification. Thirteen ATP problems were solved by inspecting the original Mizar proofs and pruning unnecessary axioms by hand. It turns out that the 3 remaining ATP problems (all coming from checking the derivation of Theorem 29 in Mizar article YELLOW_0) might be countersatisfiable. This was traced back to a very rare case when the MPTP algorithm for addition of background formulas was incomplete. Once one additional background formula was added manually to these three ATP problems, all of them became easily provable. Also when this additional background formula was added to the original MPTP Challenge problem, it became solvable directly by some ATP systems. This means that the original challenge problem is very likely countersatisfiable – this would have to be confirmed by finding a model. The original Mizar proof cannot be used to guide a proof due to the missing background formula, and the ATP problem does not seem to have alternative solution without it.

In summary, 251 MPTP Challenge problems' MML proofs were verified, i.e., it has been verified that the problem conjectures really are theorems of the problem axioms. Detailed documentation of all the steps of the verification process described here is available online.¹⁰

4.2. Verification of the 245 MML root theorems

As explained in Section 1.3, the next target for verification was selected to be a set of 245 MML root theorems. The hope is that if the system can be improved to verify these advanced MML theorems, it should be already good enough to verify the rest of the MML. These 245 root theorems were chosen so as to not contain any arithmetic evaluations done inside Mizar. The reason for this is that arithmetic evaluations is only just now becoming a part of the best ATP systems, and explicit addition of arithmetic evaluations as background axioms in the ATP problems is not yet sufficiently tuned in the MPTP system.¹¹

The verification procedure was very similar to that used for the MPTP Challenge problems. A newer version (0.999) of the E prover was used in GDV, and the time limit for each obligation discharge was increased to 40s. The first GDV run found several countersatisfiable problems during the verification, i.e., proof steps that were approved by the Mizar checker, but known to be countersatisfiable in their ATP translation. All these failures were traced back to three rarely occurring errors in the MPTP translation: bad interaction of first-orderization of two higher-order Mizar constructs (schemes vs. abstract terms), incomplete handling of Mizar redefinitions of the equality predicate, and insufficient background knowledge for some existential statements (**Consider** steps). These errors were corrected in MPTP, and the GDV run repeated, giving no more countersatisfiable results. From the 8575 ATP problems corresponding to the Mizar atomic inference steps, 364 could not be solved in the 40s time limit by E. Again MaLAREa (this time using also the Vampire system¹²) was used to attack this set of problems in three differently parameterized instances. An additional 283 problems were solved this way, leaving 81 problems unsolved. This is still too many problems for manual axiom pruning, so about 10 ATP (meta-)systems were run (through **SystemOnTPTP** [24]) on these problems with a 300s timelimit, solving an additional 27 problems. So there still remained 54 atomic Mizar steps unverified by ATP, coming from 28 of the 245 MML root theorems. It is possible to attempt manual solving at this point, but preliminary tests showed that further manipulations will allow some of these problems to be solved automatically. For example, several of the problems have conjectures with several conjuncts, where each conjunct needs only a part of the problem's axioms. If the conjuncts are split into separate problems,

¹⁰<http://lipa.ms.mff.cuni.cz/~urban/MPTPChallengeVerification.tar.gz>

¹¹This is not overly selective – only about 7% of atomic inferences, in about 13% of MML theorems, rely on arithmetic.

¹²Vampire was patched to output its proofs as TPTP derivations, so that the information needed for MaLAREa's learning was available.

and the redundant axioms pruned from them (this can be attempted automatically by extending MaLAREa), the problems become much easier to solve.

5. Conclusions and future work

We have defined and implemented a translation of the Mizar natural deduction formalism to a simpler sequent-like formalism, encoded in the TPTP language (which required a mild extension to deal with assumptions). The proofs of all MML theorems have been exported in this format. The GDV verification tool has been extended to fully verify these derivations, and together with its checking of the introduced derivation leaves, and a simple “correct by construction” argument about the introduced Henkin axioms, this provides a framework for full independent verification of translated MML proofs.

The framework was tested on 252 problems selected for the MPTP Challenge, verifying all of them after correcting one incompleteness problem in the MPTP translation. The framework was then tested on 245 MML root theorems, and it was again useful for uncovering rare errors in the MPTP system (no error has been traced all the way back to Mizar so far). The MML root theorems are harder for ATP verification, and even though they already brought about improvement of several tools in the verification chain (e.g., correction of problems with E’s proof mode, patching Vampire’s output, and major work on MaLAREa), there are still 54 unverified atomic Mizar steps. These hard problems will probably lead to further development of strong push-button meta-systems, integrating several ATP systems (possibly run in different modes), like SRASS [22] and MaLAREa. It is also possible that some of these problems will be found to be countersatisfiable, either because of a bug in the MPTP or the Mizar systems.

An obvious future step is to verify the whole MML from its axioms. This will require some extensions of the MPTP system, so that the background theories are correctly introduced (most of them have to be proved in Mizar before they can be used), and not treated as additional axioms. As noted above, ATP verification of MML proofs that contain arithmetical evaluations has been avoided. The corresponding TPTP derivations do already contain the necessary arithmetical facts, however this addition still remains to be fine-tuned, and we also would like to make use of the very recent arithmetical additions to several ATP systems.

It would be generally hard for other proof assistants (such as HOL Light and Isabelle) to import and verify the translated derivations, because their automated theorem proving is quite weak. However, it is likely that these systems will be able to check the detailed proof objects created by ATP systems such as E and Otter. So other future work is to instruct ATP systems to create such detailed proof objects during the GDV verification, and use them afterwards for refining the translated derivations to the point where other proof assistants will be able to import them. A similar import of the ATP proofs back to Mizar would also be useful (a basic translator for detailed Otter proof objects has been around since

2003), to further strengthen the (as always, ultimately empirical) confidence in the translation and verification processes, and to make the ATP tools easy to use by Mizar authors.

References

- [1] G. Bancerek. On the Structure of Mizar Types. In H. Geuvers and F. Kamareddine, editors, *Proceedings of the Workshop on Mathematics, Logic and Computation*, volume 85 of *Electronic Notes in Theoretical Computer Science*, pages 69–85, 2003.
- [2] K. Claessen and N. Sorensson. New Techniques that Improve MACE-style Finite Model Finding. In P. Baumgartner and C. Fermueller, editors, *Proceedings of the CADE-19 Workshop: Model Computation – Principles, Algorithms, Applications*, 2003.
- [3] M. Davis. Obvious Logical Inferences. In Hayes P., editor, *Proceedings of the 7th International Joint Conference on Artificial Intelligence*, pages 530–531, 1981.
- [4] E. Denney, B. Fischer, and J. Schumann. An Empirical Evaluation of Automated Theorem Provers in Software Certification. *International Journal on Artificial Intelligence Tools*, 15(1):81–108, 2006.
- [5] T. Hales. Formalizing the Proof of the Kepler Conjecture. In K. Slind, A. Bunker, and G. Gopalakrishnan, editors, *Proceedings of the 18th International Conference on Theorem Proving in Higher Order Logics*, number 3223 in *Lecture Notes in Artificial Intelligence*, page 117. Springer-Verlag, 2004.
- [6] J. Harrison. A Mizar Mode for HOL. In J. von Wright, J. Grundy, and J. Harrison, editors, *Proceedings of the 9th International Conference on Theorem Proving in Higher Order Logics*, number 1125 in *Lecture Notes in Computer Science*, pages 26–30, Springer-Verlag, 1996.
- [7] S. Jaskowski. On the Rules of Suppositions. *Studia Logica*, 1, 1934.
- [8] S. McLaughlin. An Interpretation of Isabelle/HOL in HOL Light. In U. Furbach and N. Shankar, editors, *Proceedings of the 3rd International Joint Conference on Automated Reasoning*, number 4130 in *Lecture Notes in Artificial Intelligence*, pages 192–204, Springer-Verlag, 2006.
- [9] W. McCune and O. Shumsky-Matlin. Ivy: A Preprocessor and Proof Checker for First-Order Logic. In M. Kaufmann, P. Manolios, and J. Strother Moore, editors, *Computer-Aided Reasoning: ACL2 Case Studies*, number 4 in *Advances in Formal Methods*, pages 265–282, Kluwer Academic Publishers, 2000.
- [10] A. Naumowicz and C. Bylinski. Basic Elements of Computer Algebra in Mizar. *Mechanized Mathematics and Its Applications*, 2:9–16, 2002.
- [11] A. Naumowicz and C. Bylinski. Improving Mizar Texts with Properties and Requirements. In A. Trybulec, editor, *Proceedings of the 3rd International Conference on Mathematical Knowledge Management*, volume 3119 of *Lecture Notes in Computer Science*, pages 290–301, 2004.
- [12] A. Nonnengart and C. Weidenbach. Computing Small Clause Normal Forms. In A. Robinson and A. Voronkov, editors, *Handbook of Automated Reasoning*, pages 335–367, Elsevier Science, 2001.

- [13] S. Obua and S. Skalberg. Importing HOL into Isabelle/HOL. In U. Furbach and N. Shankar, editors, *Proceedings of the 3rd International Joint Conference on Automated Reasoning*, number 4130 in Lecture Notes in Artificial Intelligence, pages 298–302, Springer-Verlag, 2006.
- [14] F. J. Pelletier. A Brief History of Natural Deduction. *History and Philosophy of Logic*, 20:1–31, 1999.
- [15] Y. Puzis, Y. Gao, and G. Sutcliffe. Automated Generation of Interesting Theorems. In G. Sutcliffe and R. Goebel, editors, *Proceedings of the 19th International FLAIRS Conference*, pages 49–54, AAAI Press, 2006.
- [16] A. Quaife. Automated Deduction in von Neumann–Bernays–Gödel Set Theory. *Journal of Automated Reasoning*, 8(1):91–147, 1992.
- [17] P. Rudnicki and A. Trybulec. On Equivalents of Well-Foundedness. *Journal of Automated Reasoning*, 23(3-4):197–234, 1999.
- [18] P. Rudnicki. Obvious Inferences. *Journal of Automated Reasoning*, 3(4):383–393, 1987.
- [19] P. Rudnicki. An Overview of the Mizar Project. In *Proceedings of the 1992 Workshop on Types for Proofs and Programs*, pages 311–332, 1992.
- [20] A. Riazanov and A. Voronkov. The Design and Implementation of Vampire. *AI Communications*, 15(2-3):91–110, 2002.
- [21] S. Schulz. E: A Brainiac Theorem Prover. *AI Communications*, 15(2-3):111–126, 2002.
- [22] G. Sutcliffe and Y. Puzis. SRASS – a Semantic Relevance Axiom Selection System. In F. Pfenning, editor, *Proceedings of the 21st International Conference on Automated Deduction*, number 4603 in Lecture Notes in Artificial Intelligence, pages 295–310, Springer-Verlag, 2007.
- [23] G. Sutcliffe, S. Schulz, K. Claessen, and A. Van Gelder. Using the TPTP Language for Writing Derivations and Finite Interpretations. In U. Furbach and N. Shankar, editors, *Proceedings of the 3rd International Joint Conference on Automated Reasoning*, number 4130 in Lecture Notes in Artificial Intelligence, pages 67–81, 2006.
- [24] G. Sutcliffe. SystemOnTPTP. In D. McAllester, editor, *Proceedings of the 17th International Conference on Automated Deduction*, number 1831 in Lecture Notes in Artificial Intelligence, pages 406–410, Springer-Verlag, 2000.
- [25] G. Sutcliffe. Semantic Derivation Verification. *International Journal on Artificial Intelligence Tools*, 15(6):1053–1070, 2006.
- [26] D. Syme. Three Tactic Theorem Proving. In Y. Bertot, G. Dowek, A. Hirschowitz, C. Paulin, and L. Théry, editors, *Proceedings of the 12th International Conference on Theorem Proving in Higher Order Logics*, number 1690 in Lecture Notes in Artificial Intelligence, pages 203–220, Springer-Verlag, 1999.
- [27] G. Sutcliffe, J. Zimmer, and S. Schulz. TSTP Data-Exchange Formats for Automated Theorem Proving Tools. In W. Zhang and V. Sorge, editors, *Distributed Constraint Problem Solving and Reasoning in Multi-Agent Systems*, number 112 in Frontiers in Artificial Intelligence and Applications, pages 201–215, IOS Press, 2004.
- [28] A. Tarski. Über unerreichbare Kardinalzahlen. *Fundamenta Mathematicae*, 30:68–89, 1938.

- [29] A. Trybulec. Tarski Grothendieck Set Theory. *Journal of Formalized Mathematics*, 1(1):9–11, 1990.
- [30] J. Urban and G. Bancerek. Presenting and Explaining Mizar. In S. Autexier and C. Benz Müller, editors, *Proceedings of the 7th Workshop on User Interfaces for Theorem Provers, 3rd International Joint Conference on Automated Reasoning*, volume 174 of *Electronic Notes in Theoretical Computer Science*, pages 63–74, 2006.
- [31] J. Urban. MPTP – Motivation, Implementation, First Experiments. *Journal of Automated Reasoning*, 33(3-4):319–339, 2004.
- [32] J. Urban. XML-izing Mizar: Making Semantic Processing and Presentaion of MML Easy. In M. Kohlhase, editor, *Proceedings of the 4th International Conference on Mathematical Knowledge Management*, volume 3863 of *Lecture Notes in Computer Science*, pages 346–360, 2005.
- [33] J. Urban. MPTP 0.2: Design, Implementation, and Initial Experiments. *Journal of Automated Reasoning*, 37(1-2):21–43, 2006.
- [34] J. Urban. MaLARea: a Metasystem for Automated Reasoning in Large Theories. In J. Urban, G. Sutcliffe, and S. Schulz, editors, *Proceedings of the CADE-21 Workshop on Empirically Successful Automated Reasoning in Large Theories*, volume 257 of *CEUR Workshop Proceedings*, pages 45–58, 2007.
- [35] J. Urban, S. Trac, G. Sutcliffe, and Y. Puzis. Combining Mizar and TPTP Semantic Presentation Tools. In *Proceedings of the Mathematical User-Interfaces Workshop 2007*, 2007.
- [36] C. Weidenbach, U. Brahm, T. Hillenbrand, E. Keen, C. Theobald, and D. Topic. SPASS Version 2.0. In A. Voronkov, editor, *Proceedings of the 18th International Conference on Automated Deduction*, number 2392 in *Lecture Notes in Artificial Intelligence*, pages 275–279, Springer-Verlag, 2002.
- [37] M. Wenzel. Isar – A Generic Interpretative Approach to Readable Formal Proof Documents. In Y. Bertot, G. Dowek, A. Hirschowitz, C. Paulin, and L. Théry, editors, *Proceedings of the 12th International Conference on Theorem Proving in Higher Order Logics*, number 1690 in *Lecture Notes in Artificial Intelligence*, pages 167–184, Springer-Verlag, 1999.
- [38] F. Wiedijk. CHECKER – Notes on the Basic Inference Step in Mizar. <http://www.cs.kun.nl/~freek/mizar/by.dvi>, 2000.
- [39] V. Zammit. On the Implementation of an Extensible Declarative Proof Language. In Y. Bertot, G. Dowek, A. Hirschowitz, C. Paulin, and L. Théry, editors, *Proceedings of the 12th International Conference on Theorem Proving in Higher Order Logics*, number 1690 in *Lecture Notes in Artificial Intelligence*, pages 185–202, Springer-Verlag, 1999.

Josef Urban

Department of Theoretical Computer Science and Mathematical Logic

Faculty of Mathematics and Physics

Charles University

Malostranske namesti 2/25

118 00 Praha 1

Czech Republic

e-mail: urban@kti.mff.cuni.cz

Geoff Sutcliffe
Department of Computer Science
University of Miami
P.O. Box 248154
Coral Gables
FL 33124-4245
USA
e-mail: geoff@cs.miami.edu

Received: December 17, 2007.

Revised: March 12, 2008.

Accepted: June 14, 2008.